NIC Firmware Upgrade — ConnectX-8 (40.48 -> 40.50.1002)

Overview

Two CRs drive a firmware update via the NVIDIA NIC Configuration Operator:

- **nicfirmwaresource.yaml** (`NicFirmwareSource`) — tells the operator where to get the firmware. `binUrlSources` points at the vendor-hosted `.zip` (the operator downloads and unzips it itself); status reports `state: Success` and `binaryVersions: {"<version>": ["<PSID>"]}` once cached.
- **nicfirmwaretemplate-worker3.yaml** / **nicfirmwaretemplate-worker4.yaml** (`NicFirmwareTemplate`) — selects which NICs on which node get the update (`nodeSelector` + `nicSelector.nicType` / `pciAddresses`) and references the source CR via `template.nicFirmwareSourceRef`, with `updatePolicy: Update` to trigger it. `nodeSelector` is a plain `map[string]string` (equality match only), so one node per template CR.

Apply order: `NicFirmwareSource` first, then the `NicFirmwareTemplate`(s) — the template does nothing until the source reports `Success`.

External-BMC platforms — read this before starting

From the NIC Firmware Configuration docs:

> On platforms where the NIC is controlled by an external BMC — including NVIDIA DGX and HGX systems (GB200 NVL72, B200, B300) and the Vera Rubin GB family — an OS-level node reboot does **not** reload NIC firmware: the BMC keeps the device powered across the reboot. As a result, persistent firmware parameters set with `mlxconfig` are not applied, and the operator stack can end up in a reboot loop trying to converge on the requested configuration.
>
> To avoid this, configure both the NIC Configuration Operator and the SR-IOV Network Operator to perform an explicit firmware reset (`mlxfwreset / mstfwreset`) **before** the reboot. The two settings are independent — enable both when both operators are deployed, or just the one matching the operator you are running.

worker3/worker4 are ConnectX-8 NICs on a B300 NVL8 (external-BMC) platform, so this bit us directly: `FW_RESET_AFTER_CONFIG_UPDATE: "true"` was added to `nicConfigurationOperator` in ../network-Operator/nicclusterpolicy.yaml to make the daemon call `mlxfwreset` after burning. In practice this still wasn't sufficient on its own — `mlxfwreset` only supported reset-level 4 ("Warm Reboot") on this hardware, which just *requests* a reboot rather than performing one; the operator treated the request as success and released node maintenance before the firmware actually activated (flash `FW Version` != `FW Version(Running)`). A manual, real node reboot was required to finish activation — see the troubleshooting commands below for how to tell the difference.

Prerequisite: persistent storage for firmware caching (`nicFirmwareStorage`)

The NIC Configuration Operator needs a PVC mounted at `/nic-firmware` in its pods to download/unzip and cache firmware binaries — without it, `NicFirmwareSource` reconciliation fails outright. It's configured under `nicConfigurationOperator` in ../network-Operator/nicclusterpolicy.yaml:

```
nicFirmwareStorage:
  create: true
```

```
  pvcName: nic-fw-storage-pvc
  storageClassName: nfs-csi
  availableStorageSize: 5Gi
```

If `nicFirmwareStorage` is omitted entirely, firmware validation/upgrade is disabled outright (per the NVIDIA docs). We hit this directly: the PVC bound fine, but the underlying NFS export (`10.20.66.66:/var/nfs/shared`, via the `nfs-csi` storage class) had `root_squash` enabled, so the CSI-provisioned sub-directory for the PVC came back owned by `nobody:nobody` — unwritable by the operator pod, causing `mkdir /nic-firmware/<source-name>: permission denied`. Fix was `chmod -R 777` on the NFS export root on the NFS server itself (not just the one sub-directory, since every future PVC against that storage class would hit the same issue otherwise).

Before you start: check current PSID and firmware version

```
# List ConnectX-8 PCIe addresses on a node (vendor 15b3 = Mellanox/NVIDIA; ConnectX-8 device IDs are 101
oc debug node/worker3 -- chroot /host lspci -d 15b3: | grep -i "ConnectX-8"

# Or, once the NIC Configuration Operator has discovered devices, straight from the NicDevice CRs
# (nicType "1023" == ConnectX-8), which also gives you the node name alongside each PCI address:
oc get nicdevice -n nvidia-network-operator -o jsonpath='{range .items[?(@.status.type=="1023")]}{.metad

# Per-device PSID + current firmware, straight from the NicDevice CR (no need to touch the node)
oc get nicdevice -n nvidia-network-operator -o jsonpath='{range .items[*]}{.metadata.name}{"\t"}{.status

# Or live from hardware inside the nic-configuration-daemon pod on that node
oc exec -n nvidia-network-operator pod/<nic-configuration-daemon-pod> -- flint -d <pci-address> query |
```

Where to get firmware / docs

- Firmware download center (PSID -> version -> `.bin.zip` + MD5/SHA256): network.nvidia.com/support/firmware/connectx8
- Full NIC Firmware Configuration procedure (`NicFirmwareSource` / `NicFirmwareTemplate` CR reference, status fields, firmware mismatch notifications): nic-fw-configuration.html#configure-and-apply-the-nicfirmwaresource-cr

Commands

Tail the controller vs. the per-node daemon

`nic-configuration-operator` is the cluster-wide controller (reconcile/template-matching events); `nic-configuration-daemon` runs per-node and is where the actual burn/reset activity happens.

```
# Cluster-wide controller
oc logs -f pod/nic-configuration-operator-7fdc9fc5f6-fxqlr

# Per-node daemon on worker3
oc logs -n nvidia-network-operator pod/nic-configuration-daemon-bddw8
```

Per-device firmware version

Recorded on each worker3 ConnectX-8 `NicDevice` CR:

```
for d in $(oc get nicdevice -n nvidia-network-operator -o name | grep worker3-1023); do
  echo -n "$d : "
  oc get -n nvidia-network-operator "$d" -o jsonpath='{.status.firmwareVersion}{"\n"}'
done
```

Checking if a firmware update is in progress / stuck

1. `NicDevice` status.conditions — the authoritative source of truth. `FirmwareConfigMatch=True` + `FirmwareUpdateInProgress=False` means done and activated.

```
oc get -n nvidia-network-operator nicdevice/worker3-1023-0000-17-00 -o jsonpath='{.status.conditions}' |
```

2. Grep terms to look for in the `nic-configuration-daemon` pod logs, in the order they appear:

Log phrase	Meaning
`InstallFirmware()`	burn attempt started for a device
`Burned firmware version does not match...`	burn needed, proceeding
`Starting firmware image burning`	actual flint/mstflint burn in progress (can take a while)
`Burned firmware version already matches...skipping burn`	flash already has the target version
`failed to burn FW on device` / `Failed to run flint` / `fw image file is not bootable`	burn errors
`New firmware image was successfully burned. Firmware reset is required`	burn ok, reset queued
`ResetNicFirmware()`	`mlxfwreset` being invoked (only fires with `FW_RESET_AFTER_CONFIG_UPDATE=true` on `NicClusterPolicy`)

```
oc logs -n nvidia-network-operator pod/nic-configuration-daemon-bddw8 | grep -iE \
  "InstallFirmware\(\)|Burned firmware|Starting firmware image burning|failed to burn FW|Failed to run f
```

3. Live hardware query — ground truth, bypasses the CR's cached status, run inside the daemon pod. `FW Version` is the version currently staged in flash; `FW Version(Running)` is the version actually active/loaded, and only appears when it differs from staged (i.e. a reset/reboot is still pending).

```
oc exec -n nvidia-network-operator pod/nic-configuration-daemon-bddw8 -- flint -d 0000:17:00.0 query | g
```

3a. Find the daemon pod for a given node (name changes on every pod restart/reboot):

```
NEWPOD=$(oc get pods -n nvidia-network-operator -l app.kubernetes.io/name=nic-configuration-daemon --fie
echo "$NEWPOD"
```

3b. Query every ConnectX-8 PCIe function on that node in one pass — if any line shows a second `FW Version(Running)` value, that device is still waiting on an activation reset/reboot:

```
for pci in 0000:17:00.0 0000:39:00.0 0000:5f:00.0 0000:70:00.0 0000:97:00.0 0000:b9:00.0 0000:dc:00.0 00
  echo -n "$pci : "
  oc exec -n nvidia-network-operator pod/$NEWPOD -- flint -d "$pci" query 2>&1 | grep -iE 'FW Version' |
  echo
done
```

4. If it looks stuck in a burn/reset loop, check whether maintenance was released prematurely:

```
oc get nodemaintenance -n nvidia-network-operator -o wide
oc get node worker3 -o jsonpath='{.spec.unschedulable}{"\n"}'
```

Appendix: YAML files used in this upgrade

NicClusterPolicy

```
apiVersion: mellanox.com/v1alpha1
kind: NicClusterPolicy
metadata:
  name: nic-cluster-policy
spec:
  docaTelemetryService:
    image: doca_telemetry
    imagePullSecrets: []
    repository: nvcr.io/nvidia/doca
    version: 1.26.5-doca3.5.0-host
  nicConfigurationOperator:
    configurationDaemon:
      image: nic-configuration-operator-daemon
      imagePullSecrets: []
      repository: nvcr.io/nvidia/mellanox
      version: network-operator-v26.7.0
    env:
      - name: SKIP_DEVICE_ON_DISCOVERY_ERROR
        value: 'true'
      # External-BMC platforms (B200/B300/GB200/GB300 etc.) keep the NIC powered across an
      # OS reboot, so a flashed image never activates without an explicit mlxfwreset first.
      - name: FW_RESET_AFTER_CONFIG_UPDATE
        value: 'true'
    logLevel: info
    operator:
      image: nic-configuration-operator
      imagePullSecrets: []
      repository: nvcr.io/nvidia/mellanox
      version: network-operator-v26.7.0
    nicFirmwareStorage:
      create: true
      pvcName: nic-fw-storage-pvc
      storageClassName: nfs-csi
      availableStorageSize: 5Gi
```

NicFirmwareSource

```
apiVersion: configuration.net.nvidia.com/v1alpha1
kind: NicFirmwareSource
metadata:
  name: connectx8-firmware-40-50-1002
  namespace: nvidia-network-operator
  finalizers:
    - configuration.net.nvidia.com/nic-configuration-operator
spec:
  # Operator unzips this automatically before flashing; PSID P6612 / OPN 900-9X86E-00CX-STO family (Conn
  binUrlSources:
    - https://www.mellanox.com/downloads/firmware/fw-ConnectX8-rel-40_50_1002-P6612_Ax-UEFI-14.43.10-Fle
```

NicFirmwareTemplate — worker3

```
apiVersion: configuration.net.nvidia.com/v1alpha1
kind: NicFirmwareTemplate
metadata:
  name: cx8-firmware-upgrade-worker3
  namespace: nvidia-network-operator
spec:
  nicSelector:
    nicType: '1023'  # 1023 points to ConnectX-8
    pciAddresses:
      - '0000:17:00.0'
      - '0000:17:00.1'
      - '0000:39:00.0'
      - '0000:39:00.1'
      - '0000:5f:00.0'
      - '0000:5f:00.1'
      - '0000:70:00.0'
      - '0000:70:00.1'
      - '0000:97:00.0'
      - '0000:97:00.1'
      - '0000:b9:00.0'
      - '0000:b9:00.1'
      - '0000:dc:00.0'
      - '0000:dc:00.1'
      - '0000:ed:00.0'
      - '0000:ed:00.1'
  nodeSelector:
    kubernetes.io/hostname: worker3
  template:
    nicFirmwareSourceRef: connectx8-firmware-40-50-1002  # References the Source CR name
    updatePolicy: Update  # Triggers the Operator to apply the update
```

NicFirmwareTemplate — worker4

```
apiVersion: configuration.net.nvidia.com/v1alpha1
kind: NicFirmwareTemplate
metadata:
  name: cx8-firmware-upgrade-worker4
  namespace: nvidia-network-operator
spec:
  nicSelector:
    nicType: '1023'  # 1023 points to ConnectX-8
    pciAddresses:
```

```
      - '0000:17:00.0'
      - '0000:17:00.1'
      - '0000:39:00.0'
      - '0000:39:00.1'
      - '0000:5f:00.0'
      - '0000:5f:00.1'
      - '0000:70:00.0'
      - '0000:70:00.1'
      - '0000:97:00.0'
      - '0000:97:00.1'
      - '0000:b9:00.0'
      - '0000:b9:00.1'
      - '0000:dc:00.0'
      - '0000:dc:00.1'
      - '0000:ed:00.0'
      - '0000:ed:00.1'
  nodeSelector:
    kubernetes.io/hostname: worker4
  template:
    nicFirmwareSourceRef: connectx8-firmware-40-50-1002  # References the Source CR name
    updatePolicy: Update  # Triggers the Operator to apply the update
```